

Question 1. [6 MARKS] Basic Python (5 minutes)

In the box beside each of the code snippets below, write the **type** of the value stored in **var**, or write “Error” if the code would result in an error. Recall that the basic Python types we used are **int**, **float**, **bool**, **str**, **list**, and **dict** (for dictionary).

<code>var = "99"</code>	string
<code>var = 3.0 / 2.0</code>	float
<code>var = ["99"]</code>	list
<code>L = ["99", "name", True, [1, 2], 3]</code> <code>var = L[2]</code>	bool
<code>L = ["99", "name", True, [1, 2], 3]</code> <code>var = L[-1]</code>	int
<code>L = ["99", "name", True, [1, 2], 3]</code> <code>var = L[2:]</code>	list

Question 2. [6 MARKS] Python Multiple Choice Questions (10 minutes)**Part (a)** [1 MARK] Select the option below that is printed when the following two lines are run.

```
name = "Ada"  
print(name * 3)
```

- ☐ An error message because strings can't be multiplied.
- ☐ 9
- ☐ Ada3
- ☒ AdaAdaAda

Part (b) [1 MARK] Select the option below that describes the contents of `var` in the line below.

```
var = "April is the cruellest month, breeding / Lilacs out of the dead land".split()
```

- ☐ A list of characters
- ☒ A list of words split on spaces
- ☐ A list of two phrases split only at /
- ☐ An error message because `.split()` needs an argument.

Part (c) [1 MARK] What is the result of the following code?

- ☐ 6
- ☐ 'abbccc'
- ☒ 3
- ☐ 'a bb ccc'

Part (d) [1 MARK]

A prof reports the results of a midterm to a class. They say that the average (mean) was 65%, but the median was 72%. Select **all** of the following statements that are true about this scenario.

- ☒ Because the average is lower than median there must be some very low marks.
- ☐ Because the average is lower than median there must be quite a few students with nearly perfect marks.
- ☒ More than half of the students got a mark higher than 65%.
- ☐ More than half of the students got a mark higher than 72%.

Part (e) [1 MARK] Select the option below that shows what the following code will print.

```
x = 10
if x > 10:
    print("A")
elif x > 3:
    print("B")
else:
    print("C")
```

- ☐ A
- ☒ B
- ☐ A and B
- ☐ C
- ☐ Nothing will print

Part (f) [1 MARK] Select the option below that gives the evaluation of this expression.

```
not False or False
```

- ☒ True
- ☐ False
- ☐ not
- ☐ Error

Question 3. [6 MARKS] (15 minutes)**Part (a)** [1 MARK]

Suppose a variable `phone_num` contains a phone number as a string in the format "416-999-7777" where each component of the phone number is separated by a hyphen. Write one or more lines of python to store the area code in a variable named `area_code`. The area code is the first three digits before the first hyphen ("-").

SOLUTION

```
# Some possible correct answers
area_code = phone_num[:3]
area_code = phone_number.split("-")[0]

data = phone_num.split("-")
area_code = data[0]
```

Part (b) [5 MARKS]

Write several lines of python code that will store a list of unique area codes (the first three digits) from the list of phone numbers in `phone_list` in a variable called `unique_codes`. You can treat these numbers as strings rather than integers.

Example: Given the following values for `phone_list`

```
phone_list = ["416-999-7777", "647-999-1234", "416-888-1111",
              "437-789-6655", "905-656-5333", "905-999-8765"]
```

the value of `unique_codes` will be (the order doesn't matter)

```
["416", "647", "437", "905"]
```

SOLUTION

```
unique_codes = []
for phone_number in phone_list:
    area_code = phone_number.split("-")[0]
    if area_code not in unique_codes:
        unique_codes.append(area_code)
```